



Principle and solutions



- ▶ Filesystems are used to organize data in directories and files on storage devices or on the network. The directories and files are organized as a hierarchy
- ▶ In UNIX systems, applications and users see a **single global hierarchy** of files and directories, which can be composed of several filesystems.
- ▶ Filesystems are **mounted** in a specific location in this hierarchy of directories
 - When a filesystem is mounted in a directory (called *mount point*), the contents of this directory reflect the contents of this filesystem.
 - When the filesystem is unmounted, the *mount point* is empty again.
- ▶ This allows applications to access files and directories easily, regardless of their exact storage location



Filesystems (2)



mount / umount

- ▶ **mount** allows to mount filesystems
 - `mount -t type device mountpoint`
 - `type` is the type of filesystem (optional for non-virtual filesystems)
 - `device` is the storage device, or network location to mount
 - `mountpoint` is the directory where files of the storage device or network location will be accessible
 - `mount` with no arguments shows the currently mounted filesystems
- ▶ **umount** allows to unmount filesystems
 - This is needed before rebooting, or before unplugging a USB key, because the Linux kernel caches writes in memory to increase performance. `umount` makes sure that these writes are committed to the storage.



Root filesystem

- ▶ A particular filesystem is mounted at the root of the hierarchy, identified by `/`
- ▶ This filesystem is called the **root filesystem**
- ▶ As `mount` and `umount` are programs, they are files inside a filesystem.
 - They are not accessible before mounting at least one filesystem.
- ▶ As the root filesystem is the first mounted filesystem, it cannot be mounted with the normal `mount` command
- ▶ It is mounted directly by the kernel, according to the `root=` kernel option
- ▶ When no root filesystem is available, the kernel panics:

```
Please append a correct "root=" boot option Kernel panic - not syncing: VFS: Unable to mount root fs on  
unknown block(0,0)
```



Location of the root filesystem

- ▶ It can be mounted from different locations
 - From the partition of a hard disk
 - From the partition of a USB key
 - From the partition of an SD card
 - From the partition of a NAND flash chip or similar type of storage device
 - From the network, using the NFS protocol
 - From memory, using a pre-loaded filesystem (by the bootloader)
 - etc.
- ▶ It is up to the system designer to choose the configuration for the system, and configure the kernel behavior with `root=`



Mounting rootfs from storage devices

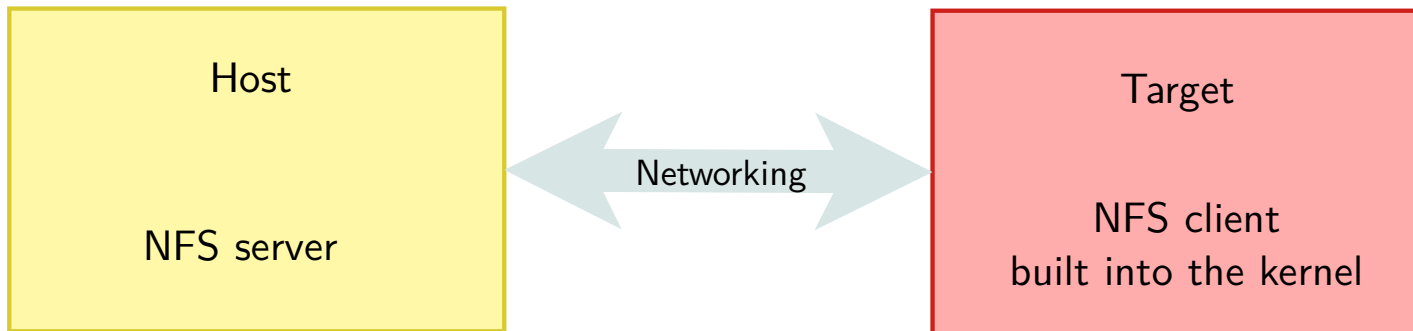
- ▶ Partitions of a hard disk or USB key
 - `root=/dev/sdXY`, where `X` is a letter indicating the device, and `Y` a number indicating the partition
 - `/dev/sdb2` is the second partition of the second disk drive (either USB key or ATA hard drive)
- ▶ Partitions of an SD card
 - `root=/dev/mmcblkXpY`, where `X` is a number indicating the device and `Y` a number indicating the partition
 - `/dev/mmcblk0p2` is the second partition of the first device
- ▶ Partitions of flash storage
 - `root=/dev/mtdblockX`, where `X` is the partition number
 - `/dev/mtdblock3` is the fourth enumerated flash partition in the system (there could be multiple flash chips)



Mounting rootfs over the network (1)

Once networking works, your root filesystem could be a directory on your GNU/Linux development host, exported by NFS (Network File System). This is very convenient for system development:

- ▶ Makes it very easy to update files on the root filesystem, without rebooting.
- ▶ Can have a big root filesystem even if you don't have support for internal or external storage yet.
- ▶ The root filesystem can be huge. You can even build native compiler tools and build all the tools you need on the target itself (better to cross-compile though).





Mounting rootfs over the network (2)

On the development workstation side, a NFS server is needed

- ▶ Install an NFS server (example: Debian, Ubuntu) `sudo apt install nfs-kernel-server`
- ▶ Add the exported directory to your `/etc/exports` file: `/home/tux/rootfs`
`192.168.1.111(rw,no_root_squash,no_subtree_check)`
 - `192.168.1.111` is the client IP address
 - `rw,no_root_squash,no_subtree_check` are the NFS server options for this directory export.
- ▶ Ask your NFS server to reload this file: `sudo exportfs -r`



Mounting rootfs over the network (3)

- ▶ On the target system
- ▶ The kernel must be compiled with
 - `CONFIG_NFS_FS=y` (NFS **client** support)
 - `CONFIG_ROOT_NFS=y` (support for NFS as rootfs)
 - `CONFIG_IP_PNP=y` (configure IP at boot time)
- ▶ The kernel must be booted with the following parameters:
 - `root=/dev/nfs` (we want rootfs over NFS)
 - `ip=192.168.1.111` (target IP address)
 - `nfsroot=192.168.1.110:/home/tux/rootfs/` (NFS server details)
 - You may need to add “`,nfsvers=3,tcp`” to the `nfsroot` setting, as an NFS version 2 client and UDP may be rejected by the NFS server in recent GNU/Linux distributions.



Mounting rootfs over the network (4)

